

Concepts :

1. Website Deployment
2. DNS, Custom Domain
3. Scaling
4. Global Deployment
5. CDN
6. Load Balancing
7. Firewall Rules
8. Networking, IPs, Ports, Subnets, IG, Routing
9. IAC
10. Manage Containers
11. Deploy Full Stack Webapp
12. ETC
13. SSL/TLS Working

Topics

- What is virtualization
- What is cloud computing
- What is aws
- Working with aws services
 - ~ aws setup lab
 - ~ aws services like ec2, s3, rds, vpc, route 53 and so on.

Virtualization and Virtual Machine

- Virtualization is the process of creating multiple simulated environments or virtual machines from a single physical hardware system, enabling more efficient resource use.

- Hypervisor : It is a software that creates and runs virtual machines (VMs).

- How hypervisor works ?

1. virtual box share hardware resources from host os
2. separate set of virtual cpu, ram, storage, etc
3. vms are fully isolated (independent of hosted os)

- Benefits :

- a. we don't need new resources to use different os
- b. no risk of any issues with your primary os
- c. testing any app on different os

- Types of hypervisor :

1. Type 1 (Bare Metal)
 - setup over a hardware
 - vmware vsphere / esxi
 - xen / citrix xenserver
 - used by cloud providers and enterprise servers
2. Type 2 (Hosted)
 - setup over a host system

- Virtualization use case :

- a. cheap
- b. reduces workload
- c. reduces space
- d. reduces energy
- e. easy backup using snapshot
- f. easy recovery

Cloud Computing

- On demand delivery of IT resources over the internet with pay-as-you-go pricing.
- Access computing resources (like servers, storage, databases and software) over the internet, rather than owning and maintaining physical hardware.
- Companies providing cloud computing :
 1. Amazon Web Services (AWS)
 2. Microsoft Azure
 3. Google Cloud Platform (GCP)
 4. IBM Cloud
 5. Oracle Cloud
 6. Alibaba Cloud
 7. Digital Ocean
 8. Salesforce
 9. VMware Cloud
 10. Rackspace Cloud
- Types of cloud computing :
 1. IaaS (Infrastructure as a Service)
 - IaaS contains the basic building blocks for cloud IT.
 - It typically provides access to networking features, computers (virtual or on dedicated hardware) and data storage space.
 - IaaS gives you the highest level of flexibility and management control over your IT resources.
 - It is most similar to the existing IT resources with which many IT departments and developers are familiar.
 2. PaaS (Platform as a Service)
 - PaaS removes the need for you to manage underlying infrastructure (usually hardware and operating systems), and allows you to focus on the deployment and management of your applications.
 - This helps you be more efficient as you don't need to worry about resource procurement, capacity planning, software maintenance, patching or any of the other undifferentiated heavy lifting involved in running your application.
 3. SaaS (Software as a Service)
 - SaaS provides you with a complete product that is run and managed by the service provider.
 - In most cases, people referring to SaaS are referring to end-user applications (such as web based email).
 - With a SaaS offering, you don't have to think about how the service is maintained or how the underlying infrastructure is managed.
 - You only need to think about how you will use that particular software.
- Different types of cloud deployments :
 - a. Public cloud - a shared cloud environment where multiple users can access services over the internet, like AWS or Azure.
 - b. Private cloud - a dedicated cloud environment for one organization, offering more control and privacy.
 - c. Hybrid cloud - a mix of public and private clouds, allowing data and applications to move between them for flexibility.
- Features commonly offered by cloud providers :

* Compute services	* Storage services
* Database services	* Container services
* Networking services	* Serverless computing
* Machine learning services	* Identity and access management (IAM)
* Monitoring and logging	* Content delivery network (CDN)
* Backup and disaster recovery	* Security and compliance tools
* Virtual private cloud (VPC)	* Data analytics services
* API management	

Amazon Web Services

- AWS offers a vast range of services, including compute power, storage solutions, networking, databases and much more.
- It offers a pay as you go model, allowing you to pay only for what you use, which is ideal for optimizing costs.
- AWS was launched in 2006 as the first public cloud platform.
- Started with Amazon S3 for storage and EC2 for compute power. But now expanded into offering over 200 fully featured services across various domains like AI, machine learning, IoT and more.
- AWS is special because of scalability, global reach, reliability and security. It also has the largest market share.
- Most popular services provided by AWS include EC2 for scalable compute capacity, S3 for highly reliable storage, RDS for managed databases, Lambda for serverless computing and CloudFront for content delivery.
- Job roles :
 - * Cloud Engineer
 - * Solutions Architect
 - * Devops Engineer
 - * Cloud Developer
 - * Data Engineer
 - * Cloud Security Specialist
 - * Machine Learning Engineer
 - * Cloud Consultant
 - * AWS Support Engineer / Cloud Support Associate
 - * Site Reliability Engineer (SRE)

AWS Account Creation

AWS IAM Service

- IAM stands for Identity and Access Management.
- IAM is a service that helps you securely control access to AWS resources.
- It allows you to manage users, roles and permissions to define who can access what within your AWS environment.
- It is a free service and offered at no additional cost.
- It is a global service.
- Root account created by default, shouldn't be used or shared.

- Things we can do in IAM service :
 1. Create Users : You can create individual user accounts for people who need access to your AWS resources.
 2. Assign Permissions : You can assign specific permissions to users, groups or roles to control what actions they can perform on AWS services.
 3. Create Groups : You can group users together and assign permissions to the group, making management easier for multiple users.
 4. Create Roles : You can create roles to assign temporary permissions to AWS services or users, especially useful for securely managing permissions across different AWS resources.
 5. Define Policies : You can create and attach custom policies to define fine grained permissions for controlling access to AWS resources.
 6. Manage Federated Access : IAM allows integrating with external identity providers (like Active Directory) for centralized management of user access across AWS.

- MFA (Multi-Factor Authentication) is an extra layer of security that requires users to provide two or more forms of verification, like a password and a code from their phone, to access their accounts.
- MFA = username + password + security code

- Ways of accessing AWS :
 - * The AWS Management Console provides a graphical, web based approach.
 - * The AWS CLI provides a command line, scripting approach.
 - * AWS SDKs and APIs offer programmatic, code based access, allowing users to integrate AWS directly into their applications.

- Setup AWS CLI on Windows
- Setup AWS CLI on Linux
- Setup AWS CLI on Mac

- Configure AWS CLI on System

- Best Practices :
 - * Avoid using root account except of account setup.
 - * Add user to a group and assign permission to group.
 - * Use password policy or MFA.
 - * Use access keys for cli or sdk.
 - * Never share access keys or password.
 - * Audit the permission using IAM credential report.

AWS EC2 Service

- EC2 stands for Elastic Compute Cloud.

- AWS EC2 is a cloud service that provides resizable virtual servers, called instances, which you can use to run applications.

- Imagine you're running a business and need a server to host your website or application. Now instead of buying and managing physical servers, AWS EC2 lets you rent virtual servers in the cloud. These virtual servers are called instances.

- We can do various configurations like :

- * os
- * ram
- * cpu
- * disk space
- * network/firewall
- * how to access the machine

- Key terms :

- * Instance Type - Select the hardware capacity (cpu, memory).
- * AMI (Amazon Machine Image) - Choose the operating system and software (linux, mac, windows).
- * Storage - Configure the type and size of storage (ebs volume).
- * Security Groups - Setup firewall rules to control inbound/outbound traffic.
- * Key Pair - Create or use an existing key pair for SSH access.
- * Network Settings - Configure VPC, subnet, and assign public or private IP addresses.
- * IAM Role - Attach an IAM role for permissions to access other AWS resources.
- * User Data - Add scripts to be executed when the instance starts.
- * Elastic IP - Optionally associate a static IP address for consistent public access.

- Creating first ec2 instance.

- Using user data.

```

```
#!/bin/bash
sudo yum update -y
```

```
Install apache web server (httpd)
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
```

```
Create a simple html file to verify web server status
echo "<html><h1>Welcome to Apache Web Server on Amazon Linux : $(hostname)</h1></html>" >
/var/www/html/index.html
```
```

- Security Groups : network firewall rules that control inbound and outbound traffic for instances.

- Important points about security groups :

- * Region specific
- * Only allow rule and no deny rule
- * All inbound traffic blocked and outbound allowed by default
- * You define rules for specific :
 - a. protocols like HTTP, HTTPS, SSH, etc.
 - b. port numbers like 80 for HTTP, 22 for SSH, etc.
 - c. ip addresses or ranges
- * If you allow incoming traffic on a specific port like 80 for HTTP, the outgoing response traffic is automatically allowed without an explicit outbound rule.

- Some common ports :

- * HTTP (Port 80) - Unencrypted web traffic.
- * HTTPS (Port 443) - Encrypted web traffic (SSL/TLS).
- * SSH (Port 22) - Secure remote access to servers (Linux/Unix).
- * FTP (Port 21) - File transfer protocol (unsecured).
- * SFTP (Port 22) - Secure file transfer protocol.
- * SMTP (Port 25) - Simple mail transfer protocol (email sending).
- * RDP (Port 3389) - Remote desktop protocol (windows remote access).
- * MySQL (Port 3306) - MySQL database connections.
- * PostgreSQL (Port 5432) - PostgreSQL database connections.
- * DNS (Port 53) - Domain name system (converts domain names to ip addresses).

- SSH Access into EC2 Instance

- * SSH allows you to control/access a remote machine.

- EC2 Termination

- EC2 Instance Type :

1. General Purpose
2. Compute Optimized
3. Memory Optimized
4. Accelerated Computing
5. Storage Optimized
6. HPC Optimized
7. Instance Features
8. Measuring Instance Performance

- Use Cases :

- * Case 1 : Small Website or Blog
Suitable Type : t3.micro or t3.small (general purpose)
- * Case 2 : E-Commerce Application
Suitable Type : m5.large or m5.xlarge (general purpose)
- * Case 3 : Real Time Video Rendering and Streaming
Suitable Type : g5.12xlarge or g5.24xlarge (accelerated computing)
- * Case 4 : In Memory Database for Real Time Analytics
Suitable Type : r6g.16xlarge or x2idn.32xlarge (memory optimized)

- Purchasing Options :

1. On Demand :
 - * best for short term, unpredictable workloads
 - * cost is high
 - * commitment is none
 - * flexibility is high
2. Reserved Instances :
 - * best for long term, steady workloads (1-3 years)
 - * cost is medium to low (upto 75% off)
 - * commitment is 1 or 3 years
 - * flexibility is low
3. Spot Instances :
 - * best for fault tolerant, flexible, batch jobs, big data, ci/cd, distributed computing
 - * cost is very low (upto 90% off)
 - * commitment is none (subject to termination)
 - * flexibility is medium and can be interrupted
4. Savings Plans :
 - * best for consistent workloads but needing more flexibility than reserved instances
 - * cost is medium to low (upto 72% off)
 - * commitment is 1 or 3 years
 - * flexibility is high (more than RI)

5. Dedicated Hosts

- * best for workloads requiring complete hardware control (compliance, software licensing)
- * cost is high
- * commitment is 1 or 3 years
- * flexibility is medium

6. Dedicated Instances

- * best for workloads needing physical isolation without full control over the hardware
- * cost is high
- * commitment is none
- * flexibility is medium

AWS EBS Service

- EBS stands for Elastic Block Store.
- AWS EBS is a cloud based storage service that provides durable, high performance block storage for use with Amazon EC2 instances.
- It works like a virtual hard drive, allowing you to store and access data even when your EC2 instances are stopped or terminated.
- Use case : For example, if you're hosting a MySQL or PostgreSQL database, you need reliable, high-performance storage to handle frequent read/write operations. EBS provides persistent, fast storage that ensures your data is saved even if the EC2 instance is stopped or restarted, making it ideal for database workloads.
- Key points about EBS :
 - * Region and availability zone specific
 - * Built in redundancy - EBS volumes are automatically replicated within the same availability zone to prevent data loss due to hardware failures.
 - * Different volume types like gp2/3, io1/2, st1, sc1
 - * Allow encryption and snapshot for backup
 - * Scalable (volume can be resizable) - No data loss will occur during resizing and no need to restart the ec2 instance during the process.
- Delete on termination
- Create ebs and attaching to ec2 instance
- Modify size of ebs volume
- Detach and delete ebs volume
- EBS Snapshot : as backup as well as used when we want to copy our data to new az or new region.
- Mount and unmount ebs.
- Copy snapshot to another region.
- EBS Encryption :
 - * The following types of data are encrypted :
 - a. data at rest inside the volume
 - b. all data moving between the volume and the instance
 - c. all snapshots created from the volume
 - d. all volume created from those snapshots
- EBS Lifecycle Manager
- Snapshot Recycle Bin

AWS AMI Service

- AMI stands for Amazon Machine Image.

- An AMI is a pre-configured template that provides the necessary information to launch an EC2 instance in AWS.

- It includes :

- a. Operating system (linux, windows)
- b. Application server (apache, nginx)
- c. Pre installed software and configurations

- Create image

- With an AMI, you can launch new EC2 instances with a consistent, predefined configuration.

- You can also create custom AMIs to include specific software or settings, allowing for quick replication of environments.

- Types of AWS AMIs :

1. Public AMIs - available to all aws users. useful for basic use cases like popular operating systems (ubuntu, centos).
2. Private AMIs - created by a user and only available within that account or shared with specific accounts.
3. Paid AMIs / Marketplace AMIs - provided by third parties through aws marketplace, offering software like databases, web servers or pre configured environments.

- Use case of paid AMIs :

* Rapid Deployment : the lamp stack is pre configured, eliminating the need to manually install and configure apache, mysql and php.

* Scalability and Load Balancing : running on aws enables quick scaling to match website traffic, while elastic load balancer helps in distributing requests.

* Cost Efficiency : you only pay for the infrastructure and the software according to your usage.

- Run cleanup script :

```
#!/bin/bash
```

```
# Remove ssh keys
```

```
rm -rf ~/.ssh/authorized_keys
```

```
# Clean user credentials and history
```

```
rm -rf ~/.aws/credentials
```

```
rm -rf ~/.git-credentials
```

```
rm -rf ~/.bash_history
```

```
# Clean system logs and temporary files
```

```
rm -rf /var/log/*
```

```
rm -rf /var/tmp/*
```

```
rm -rf /tmp/*
```

```
# Remove user accounts
```

```
deluser tempuser --remove-home
```

```
# Lock root account
```

```
passwd -l root
```

```
# Reset configuration files like nginx
```

```
rm -rf /etc/nginx/nginx.conf
```

- Create launch template vs create image :

* Create launch template creates a reusable blueprint for launching instances whereas Create image creates a custom AMI snapshot of an instance.

- * Create launch template contains configuration settings like AMI, instance type, security groups etc whereas Create image captures os, applications, configurations and data.
- * Create launch template is used repeatedly to launch new instances in a consistent way where Create image is used to create new instances that replicate the captured image.
- * Create launch template is used with auto scaling, spot instances, standardizing instance settings whereas Create image is used for backup, replication, migrating instances to another region.
- * Create launch template can have multiple versions for different configurations while Create image have an AMI which is a point in time caputer of an instance.

- Notes :

- * Yes, all installed applications, configuration settings, environment variables, network configurations, DNS settings, users and firewall settings will be included in the AMI.
- * An AMI is essentially a complete snapshot of the instance at the point in time you created the image, allowing you to replicate the exact state of that server, including all software, configuration and operating system level changes.
- * When you create an EC2 instance from this AMI, it will boot up as if it were an exact clone of the original, with all installed software and settings intact.

- EC2 Image Builder

- * Automate vm or image creation : creation, testing and deployment of AMIs
- * Can be configured to run at regular intervals (daily, weekly, monthly)
- * Free

AWS ELB Service

- ELB stands for Elastic Load Balancing.

- Scalability : The ability to grow your system's resources when your application or website gets more traffic or more users.

- * Vertical Scalability (Scaling Up) - means adding more power (cpu, ram) to your existing server. ex : t2.micro to m5.large

- * Horizontal Scalability (Scaling Out) - means adding more instances (servers) to distribute the load. you can add more ec2 instances behind a load balancer.

- High Availability (HA) : It means keeping your service up and running with minimal downtime, so it's always accessible to users. ex : running resources in multiple AZs.

- Elasticity : It means the ability to automatically adjust resources as the demand changes by adding more when needed and removing when it is no longer necessary. ex : ASG

- Create the setup :

1. Setup ec2 instances - create two or more ec2 instances, install a web server and tag them for easy identification.
2. Configure security groups - setup a security group allowing http and ssh access.
3. Create the load balancer - use the ec2 dashboard to create an application load balancer and set it as internet facing.
4. Register targets - add ec2 instances to the target group and configure health checks.
5. Test the load balancer - access the dns name of the load balancer and observe load balancing in action.
6. Explain key concepts like traffic distribution, high availability and scalability.

- Load balancer types :

a. Application Load Balancer :

- * choose an application load balancer when you need a flexible feature set for your applications with HTTP and HTTPS traffic.

- * it is perfect for web applications, handling complex http and https requests (layer 7).

- * operating at the request level, application load balancers provide advanced routing and visibility features targeted at application architectures, including microservices and containers.

- * how application load balancers work -

1. clients make requests to your application.
2. the listeners in your load balancer receive requests matching the protocol and port that you configure.
3. the receiving listener evaluates the incoming request against the rules you specify, and if applicable, routes the request to the appropriate target group. you can use an http listener to offload the work of tls encryption and decryption to your load balancer.
4. healthy targets in one or more target groups receive traffic based on the load balancing algorithm, and the routing rules you specify in the listener.

b. Network Load Balancer :

- * choose a network load balancer when you need ultra high performance, tls offloading at scale, centralized certificate deployment, support for udp, and static ip addresses for your applications.

- * designed for high performance and low latency, perfect for tcp/udp traffic (layer 4). ex : gaming, financial apps.

- * operating at the connection level, network load balancers are capable to handling millions of requests per second securely while maintaining ultra low latencies.

- * how network load balancers work -

1. clients make requests to your application.
2. the load balancer receives the request either directly or through an endpoint for private connectivity (through aws privatelink).
3. the listeners in your load balancer receive requests of matching protocol and port, and route these requests based on the default action that you specify. you can use a tls listener to offload the work of encryption and decryption to your load balancer.
4. healthy targets in one or more target groups receive traffic according to the flow hash algorithm.

c. Gateway Load Balancer :

* choose a gateway load balancer when you need to deploy and manage a fleet of third party virtual appliances that support GENEVE.

* it helps to deploy, scale and manage third party virtual appliances, such as firewalls and monitoring solutions.

* these appliances enable you to improve security, compliance and policy controls.

* how gateway load balancers work -

1. clients make requests to your application.

2. the load balancer receives the request based on the route table configurations that are set within your vpc, internet gateway or transit gateway.

3. the load balancer routes requests to a target group consisting of a scalable fleet of appliances (for example, firewalls, deep packet inspection systems, url filtering systems etc.) to process traffic flows.

4. the virtual appliance processes the traffic, and forwards it back to the load balancer, or drops the traffic based on its configuration. this type of load balancer acts as a bump in the wire between the source and destination.

5. the load balancer forwards the traffic to its destination.

d. Classic Load Balancer :

* choose a classic load balancer when you have an existing application running in the ec2 classic network.

* classic load balancers can't route to target groups.

- Key points :

* Distributed traffic : it splits incoming traffic across multiple servers so no single server gets overloaded.

* Improves availability : if one server goes down, the load balancer automatically sends traffic to the working servers, ensuring your application stays available.

* Scale resources : it helps manage high demand by adding more servers during peak times and distributing the load.

* Single point of access need to be expose.

* High availability across availability zones.

AWS ASG Service

- ASG stands for Auto Scaling Groups.
- It is a service that automatically adds or removes ec2 instances based on demand to ensure your application is always available.
- It helps scale up when more capacity is needed and scale down during low usage to save costs, keeping the right number of servers running at all times.
- Functions :
 - * Automatic Scaling : scale the number of ec2 instances up or down based on demand.
 - * Maintain Instance Health : replace unhealthy instances automatically to ensure reliability.
 - * Use Scaling Policies : set rules for scaling based on metrics like cpu usage or request count.
 - * Ensure availability : always keep a defined number of instances running to meet application needs.
 - * Schedule scaling : pre configure scaling activities for specific times. ex : traffic peaks.
 - * Distribute instances : deploy instances across multiple availability zones for high availability.
 - * Integrate with ELB : attach instances to an elastic load balancer to automatically balance traffic.
 - * Optimize costs : scale down during low demand to save on infrastructure costs.
- Steps to create ASG :
 1. AMI, launch template and configuration.
 2. Create auto scaling group.
 3. Select vpc and subnets.
 4. Attach load balancer (optional).
 5. Configure scaling policies.
 6. Health checks.
 7. Add notifications (optional).
 8. Review and create.

AWS S3 Bucket

- S3 stands for Simple Storage Service.

- Topics :

- * S3 versioning
- * S3 replication
- * Data encryption
- * S3 bucket policies
- * S3 storage classes
- * Logging monitoring
- * Hosting static website
- * Snow family
- * Storage gateway (hybrid solution)

- S3 is a cloud based service that allows you to store, manage and retrieve large amounts of data like files, images, videos and backups securely and at scale.

- It provides highly reliable, scalable object storage, making your data accessible from anywhere, anytime via the internet.

- Key points :

- * store data as objects
- * globally unique name
- * region specific
- * each object within a bucket is stored as a key value pair where key is the object's name (contain slashes mimicking directory structure) and value is the content of the object (file or data itself)
- * 5 TB is the max size for a single object in amazon s3
- * multipart upload is recommended for objects larger than 5 GB (split the file into smaller parts and upload them separately).

- S3 Bucket Policies :

- * JSON based access control policies that you attach directly to an S3 bucket to manage permissions for accessing the bucket and its objects.
- * They allow you to define who can access the data and what actions they can perform, such as read, write or delete enabling fine grained control over the security of your data stored in s3.
- * Write or paste your JSON policy in the bucket policy editor.
- * You can use aws's policy generator to create a custom policy, or you can manually write the policy in json format.
 - a. GetObject : used to retrieve or download files from an s3 bucket.
 - b. PutObject : used to upload or add files into an s3 bucket.

- Create bucket.

- Host static website.

- S3 Versioning :

- * It allows you to keep multiple versions of an object in the same bucket, providing protection against accidental deletions or overwrites.
- * When versioning is enabled, s3 stores every version of an object, allowing you to recover older versions if needed, making it ideal for data safety and backup.

- S3 Replication :

- * It allows you to automatically copy objects from one S3 bucket to another, which can be :
 - a. within the same region (Same Region Replication - SRR)
 - b. in different regions (Cross Region Replication - CRR)
- * It is commonly used for compliance, redundancy and to improve data access performance by maintaining copies closer to your users.

- S3 Storage Classes :

- * S3 Standard - frequently accessed data
- * S3 Intelligent Tiering - data with unknown or unpredictable access patterns
- * S3 Standard IA - infrequently accessed but quickly retrievable
- * S3 One Zone IA - non critical, infrequently accessed data
- * S3 Glacier - Archival data, rarely accessed
- * S3 Glacier Deep Archive - Deep archival data, almost never accessed
- * S3 Outpost - Local storage using aws outposts.

- S3 Bucket Lifecycle :

* You can use lifecycle policies to control the movement of objects between different storage classes or delete them entirely, based on specific conditions like age or inactivity.

- S3 Snow Family :

* The s3 snow family is a group of physical devices offered by aws to help move large amounts of data to the cloud when using the internet isn't practical.

* These devices are used when there is too much data to upload over a regular connection or when dealing with remote areas without good internet.

* The snow family includes :

a. AWS Snowcone - a small, portable device for a few terabytes of data.

b. AWS Snowball - a larger device for moving petabytes of data and can also be used for edge computing.

c. AWS Snowmobile - a massive truck sized container used for exabyte scale data transfers, typically used by big companies moving entire data centers.

* These devices help you transfer data quickly, securely and cost effectively to aws, especially when internet speed or reliability is an issue.

- S3 Storage Gateway :

* It is a hybrid cloud storage service that connects on premises environments to cloud storage in amazon s3.

* It helps extend your local storage to the cloud by acting as a bridge.

* Different types of gateway :

a. Amazon S3 File Gateway

b. Amazon FSx File Gateway

c. Tape Gateway

d. Volume Gateway

AWS RDS Service

- RDS stands for Relational Database Service.
- AWS RDS as a managed database service that simplifies database setup, operation and scaling.
- It's purpose it to handle administrative tasks like backups, patching, monitoring and scaling.
- Engine types :
 - * Aurora
 - * Mysql
 - * Mariadb
 - * Postgresql
 - * Oracle
 - * Microsoft sql server
 - * IBM db2
- RDS Instance : create a rds mysql instance
 1. use free tier.
 2. username will be admin and you can set password without special characters.
 3. keep the public access to true to access it from local or remote server.
 4. create a security group (and allow 3306 from everywhere)
 5. after creating, you can find end point (hostname) to connect to this db.
- EC2 Instance :
 1. `sudo yum install -y docker`
 2. `sudo systemctl start docker`
 3. `sudo usermod -aG docker ec2-user`
 4. `sudo docker pull philippaul/node-mysql-app:02`
 5. `docker run --rm -p 80:3000`
 - e DB_HOST="your-db-hostname"
 - e DB_USER="your-db-username"
 - e DB_PASSWORD="your-db-password"
 - d philippaul/node-my-sql-app:02
 6. `docker run -it --rm mysql:8.0 mysql -h db.example.com -u admin -p`
- Aurora Offers :
 - * upto 5x the throughput of mysql community edition and 3x of postgres.
 - * upto 128 tb of autoscaling ssd storage.
 - * six way replication across three availability zones.
 - * up to 15 read replicas with replica lag under 10 ms.
 - * automatic monitoring with failover.
- Benefits of using RDS :
 - * high availability and fault tolerance.
 - * vertical and horizontal scaling.
 - * automated backups and recovery.
 - * read replicas for improved read performance.
 - * multi az setup for disaster recovery.
 - * cost effectiveness.
- RDS Read Replica
 - a. multi region
 - b. multi az
- Common use cases :
 - * web applications : relational databases are ideal for web apps requiring structured data.
 - * e-commerce platform : for handling inventory, customer data and order transactions.
 - * business applications : erp, crm and financial applications with strong data integrity needs.

AWS DynamoDB Service

- It is a fully managed, key value and document database that delivers single digit millisecond performance at any scale.
- NoSQL is a type of database designed to store and manage data in flexible, non tabular formats, making it ideal for handling large volumes of unstructured data.
- DynamoDB stores data as items in tables, with each item represented as a JSON-like document consisting of key value pairs.
- DynamoDB also offers a free tier that provides 25 GB of storage. It also includes 25 provisioned write / read capacity units (WCU, RCU) which is enough to handle 200 M requests per month.
- Features :
 - * Serverless : no need for server provisioning, software installation, maintenance or patching.
 - * Automatic scaling : instantly scales up or down based on demand, with no manual adjustments needed.
 - * Zero downtime : provides continuous availability without maintenance windows.
 - * On demand pricing : pay only for the read write requests used, ideal for fluctuating workloads.
 - * Idle cost savings : scales down to zero during inactivity, so there's no cost when tables have no traffic.
- Docker based node app with dynamodb
- It is flexible data model and reliable performance make dynamodb a great fit for mobile, web, gaming, advertising technology, internet of things and other applications.
- DAX stands for DynamoDB Accelerator.
- DAX :
 - * Fully managed in memory cache for dynamodb.
 - * DAX offers microsecond latency achieving up to a 10x performance over standard DynamoDB queries.
 - * High availability and scalability and can be deployed in multiple availability zones.
 - * DAX is only used for and is integrated with dynamodb, while elasticache can be used for other databases.
- DynamoDB Global Table

AWS Lambda Function

- AWS lambda is a serverless computing service that lets you run code in response to events without managing servers.
- You just upload your code and aws automatically handles the rest, scaling as needed and only charging for the time your code runs.
- Event Driven Execution
 - * Lambda is an event driven service, meaning that it runs your code in response to certain triggers or events.
 - * These events can come from many different aws services like :
 - a. s3 (file upload)
 - b. dynamodb (database changes)
 - c. api gateway (http requests)
 - d. cloudwatch (scheduled events), etc.
 - * Behind the scene notification is used.
- AWS Lambda Limitations :
 - * Execution time limit : lambda functions can only run for a maximum of 15 minutes. If you need longer running tasks, lambda might not be the best choice.
 - * Stateless : lambda functions don't keep state between invocations, so they're best for tasks that don't require long term memory.
 - * Cold start delays : if a lambda function hasn't run in a while, there's sometimes a slight delay called as cold start when it starts up. this can add a little latency, but aws provides ways to mitigate it for critical functions.
- Use cases :
 1. Image processing : let's say users upload images to your app. you can set up lambda to automatically resize, compress or even apply filters to each image as it's uploaded.
 2. Data transformation : if you need to clean up or process data before storing it in a database, a lambda function can handle that transformation automatically.
 3. Real time notifications : if an event happens like a new user signing up, you can use lambda to trigger and email, sms or other notifications instantly.
- Automatic scaling :
 - * AWS lambda automatically scales the execution of functions in response to the number of incoming requests.
 - * If a thousand requests come in at the same time, aws lambda will handle them in parallel, making it highly scalable without any configuration.
- Pay as you go :
 - * Lambda uses a pay as you go pricing model. You are billed based on the number of function executions and the duration of each function's runtime (measured in milliseconds).
 - * This is cost effective as you only pay for the compute time that you use and there's no charge for idle time.

AWS Cloud Formation

- AWS cloudformation is an infrastructure as code (IaC) service that lets you define, provision and manage aws resources in a declarative, template based format.

- Workflow :

1. create or use an existing template
2. save locally or in s3 bucket
3. use aws cloudformation to create a stack based on your template.
4. it constructs and configures your stack resources.

- Benefits :

- * Consistency : define infrastructure in code
- * Automation : reduce manual work
- * Repeatable : replicate environments easily

- Key points :

- * Infrastructure as Code (IaC) Tool : automates the creation, management and updating of aws infrastructure.
- * Declarative Language : users define desired end states for resources and cloud formation handles provisioning.
- * Template Based : uses yaml or json templates to specify aws resources and configurations.
- * AWS Native : exclusively supports aws resources, fully integrated with aws services.
- * State Management : manages state internally, eliminating the need for seperate state files.
- * Stacks and Stack Sets : organizes resources in stacks for easier management and allows for multi account and region deployments with stack sets.
- * Cost Free Tool : cloud formation itself is free, you only pay for the aws resources created.
- * Drift Detection : identifies and reports on resource changes made outside of cloud formation to ensure resources stay aligned with the templates.

- Use cases :

1. create ec2 instances with security groups and elastic ips.
2. provision s3 buckets with http endpoints.
3. set up vpcs with subnets and route tables.
4. create rds databases with automatic backups.
5. deploy lambda functions with api gateway integrations.
6. setup elastic load balancers and auto scaling for web applications.
7. deploy iam roles, policies and user access management.

- aws cloudformation create-stack --stack-name my-stack-name --template-body file:///path/to/template.yaml

AWS Route 53 Service

- AWS route 53 is a scalable dns service for domain registration, traffic routing and health checking capabilities.
- DNS stands for Domain Name System. It is the internet service that translates human friendly domain names like `www.example.com` into machine readable IP addresses.
- Default port for DNS (Domain Name System) service is : 53
- Practical setup
 - Global website with low latency
 - Health check
- Features :
 - a. domain names
 - b. hosted zones
 - c. health checks
 - d. traffic flow
 - e. resolver
- How route 53 works :
 1. Domain name registration : register a domain and point it to aws route 53.
 2. Hosted zone creation : create a hosted zone to manage dns records.
 3. Dns records : add records (e.g., A, CNAME, MX) to route traffic to various endpoints.
 4. Routing policies : setup routing policies based on your needs, such as latency based or failover routing.
 5. Health checks : configure health checks to monitor endpoints and trigger failover when needed.
- Types of records supported by aws route 53 :
 - * A Record (IPv4) : maps a domain name to an ipv4 address.
example - `www.google.com => 12.34.56.78`
 - * AAAA Record (IPv6) : maps a domain name to an ipv6 address.
example - `www.example.com => 2001:db8::1`
 - * CNAME Record : maps a domain name to another domain name (alias).
example - `blog.example.com => www.example.com`
 - * MX Record : directs mail to an email server.
example - `example.com => mail.example.com (priority 10)`
 - * TXT Record : provides text information to external sources for verification or configuration.
example - `example.com => "v=spf1 include:_spf.example.com ~all"`
 - * NS Record : specifies the authoritative name servers for the domain.
example - `example.com => ns-123.awsdns-45.org`
 - * SRV Record : specifies the location of services
example - `_sip._tcp.example.com => 10 60 5060 sipserver.example.com`
- Use cases :
 - a. Hosting websites : manage domain names and route traffic to web applications.
 - b. Load balancing : distribute traffic across multiple endpoints using weighted or latency based routing.
 - c. Disaster recovery : use health checks and failover routing for high availability.
 - d. Multi region deployments : route traffic to the closest region for low latency.
- Billing :
 - * Billable components include hosted zones, dns queries, health checks, domain registration and traffic policies.
 - * Costs vary based on usage and the type of configuration (e.g. standard vs advanced routing policies).
 - * Free tier : route 53 does not include a free tier, so charges start as soon as you use its services.

AWS Cloudfront CDN

- AWS cloudfront is a content delivery network (CDN) that speeds up the delivery of web content to users by caching it at servers (edge locations) close to them, improving load times and performance globally.
- AWS cloudfront primarily caches static content like images, css, javascript and videos. It can also cache dynamic content (eg. html or api responses) if configured with caching policies and headers.
- By default, sensitive or user specific data and backend logic are not cached. Cache behaviour is controlled via TTLs, cache behaviours and origin headers.
- Cloudfront has three types of infrastructure to securely deliver content with high performance to end users :
 1. CloudFront Regional Edge Caches (RECs) are situated within aws regions, between your applications web server and cloudfront points of presence (POPs) and embedded points of presence. Cloudfront has 13 RECs globally.
 2. CloudFront Points of Presence are situated with the aws network and peer with internet service provider (ISP) networks. Cloudfront has 600+ pops in 100+ cities across 50+ countries.
 3. CloudFront embedded Points of Presence are situated within internet service provider (ISP) networks, closest to end viewers. In addition to cloudfront pops, there are 600+ embedded pops across 200+ cities in north america, europe and asia.
- Browsers act like a mini-cdn by caching website files (like images, css and js) locally on user's device, which speeds up loading for repeat visits. Only helps individual users.
- Included in always free tier :
 - * 1 tb of data transfer out to the internet per month.
 - * 10,000,000 http or https requests per month.
 - * 2,000,000 cloudfront function invocations per month.
 - * 2,000,000 cloudfront key value store reads per month.
 - * free ssl certificates.
 - * no limitations, all features available.

AWS Virtual Private Cloud (VPC)

- A private, isolated network within the aws cloud where you can launch and manage your resources securely.
- VPC is needed to securely isolate and control network environments.
- VPC CIDR Block :
 - * when you create a vpc, you specify a cidr block that defines the ip address range for the entire vpc.
 - * example - 10.0.0.0/16 - this blocks allows for 65536 ip addresses but in reality 65531 usable addresses.
 - * CIDR stands for Classless Inter Domain Routing which is a method for allocating IP addresses and routing Internet Protocol (IP) packets.
 - * cidr.xyz
- Subnets :
 - * A subnet is a smaller, segmented part of a larger network that isolates and organizes devices within a specific ip address range.
 - * Resources are made inside subnet and not vpc.
 - * While creating subnet you specify a range of IP addresses (CIDR block) within the VPC's IP address range for the subnet.
 - * This determines the pool of IP addresses available for instances in the subnet.
- Understanding 10.0.1.0/24 :
 - * An ipv4 address is 32 bits long.
 - * Example in binary : `10.0.1.0` -> `00001010.00000000.00000001.00000000`
 - * The /24 indicates that the first 24 bits are the network portion of the address.
 - * The remaining 8 bits are available for host addresses within the network.
 - * 10.0.1.0 to 10.0.1.255 is the full range.
- Route table :
 - * A route table is a set of rules, called routes, that are used to determine where network traffic from your subnets or gateway is directed.
 - * Each subnet in your vpc must be associated with a route table, which controls the routing for that subnet.
- Internet gateway is a component that allows communication between instances in your vpc and the internet.
- Security groups are network firewall rules that control inbound and outbound traffic for instances.
- Network ACLs (Access Control Lists) :
 - * Optional layer of security for your vpc that acts as a firewall for controlling traffic in and out of one or more subnets.
 - * Allow or deny rule.
- NAT (Network Address Translation) Gateway : Enables instances in a private subnet to connect to the internet or other aws services, but prevents the internet for initiating connections to those instances.
- VPC Peering : A networking connection between two VPCs enabling us to route traffic between them privately.
- VPC Endpoints : Allows to privately connect VPC to supported AWS services and VPC endpoint services by AWS PrivateLink.
- Bastion Host : A special purpose instance that provides secure access to your instances in private subnets.
- Elastic IP Addresses : Static IP addresses designed for dynamic cloud computing.
- VPC Flow Logs : Capture information about the IP traffic going to and from network interfaces in your VPC.
- Direct Connect : Establishes a dedicated network connection from your premises to aws.
- AWS Client VPN : Managed vpn service that enables secure remote access to aws resources and on premises networks using openvpn based clients.

- AWS VPC Manual Creation

AWS Account and Billing Management

- Using AWS Organizations for central management of multiple aws accounts.

AWS Amplify

- Topics :
 1. Deploy static website with amplify from github.
 2. Automatic deployment
 3. Custom domain
 4. React todo app
 5. Changes in frontend and backend
 6. Amplify local env + cloud sandbox
- Amplify is a platform that simplifies :
 - a. building
 - b. deploying and
 - c. hosting full stack web and mobile apps.
- CI/CD : automated build and deployment
- Configure custom domain.
- Full Stack :
 - * Html, Css and Javascript frameworks (e.g., React, Angular, Vuejs)
 - * Python, Nodejs, Java or PHP along with frameworks such as express, django or spring boot.
 - * Mysql, MongoDB and Postgresql.
- Key points :
 - a. connects to backend services like user login, data storage and apis.
 - b. offers hosting and automatic updates for web apps.
- Amplify benefits :
 - * speeds up app development with built in tools.
 - * provides scalable backend services that can grow with your app.
 - * simplifies cloud integration, making it easier for beginners and experienced developers.
- React todo app deployment.
- Amplify cloud sandbox.
- Resource clean up.

AWS ECS

- ECS stands for Elastic Container Service.
- ECS is a cloud based container management service that allows you to run and manage docker containers on a cluster of virtual servers.
- It automatically handles :
 1. creation
 2. management
 3. updating
- Terms :
 - a. Cluster : group of tasks and services, hosts all the resources and infrastructure.
 - b. Service : handles scalability and load balancing of container.
 - c. Task : represents the running containers of your app.
- ECS -> ECS cluster (fargate) -> ECS service -> ECS task definition.

AWS EKS

- EKS stands for Elastic Kubernetes Service.
- EKS is a fully managed kubernetes cluster infrastructure.
- Kubernetes is an open source platform for automating the deployment, scaling and management of containerized applications.
 - * Automating deployment
 - * Management of containerized apps
 - * Scaling
- When you deploy kubernetes, you get a cluster.
Two important parts are :
 1. Master (control plane)
 2. Worker nodes
- Create eks cluster :
 - a. `eksctl create cluster --name=<cluster-name> --enable-auto-mode`
 - b. we can also provide
 - * `--nodes=3`
 - * `--nodes-min=2, --nodes-max=5`
 - * `--region us-west-2`
 - * `--node-type t3.medium`
 - * `--managed/fargate`
- Cluster type :
 1. EC2 Worker Nodes - customizable, performance intensive
Managed by aws : partially
 2. Fargate - serverless, lightweight workloads
Managed by aws : fully
 3. Managed Node Groups - simplified node management
Managed by aws : mostly
 4. EKS Anywhere - hybrid and on premises deployments
Managed by aws : no
 5. Spot Instances - cost sensitive, fault tolerant apps
Managed by aws : partially
 6. Outposts - on prem kubernetes workloads
Managed by aws : mostly
 7. Multi Cluster - large scale, isolated environments
Managed by aws : partially

Terraform

- Terraform is an open source infrastructure as code (IaC) tool.
- IaC enables you to define, provision and manage infrastructure across multiple cloud providers using a declarative configuration language.
- It supports multi cloud environments, allowing consistent and efficient infrastructure management.
- Example :

```
...  
  
# 1. which cloud provider  
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "5.54.1"  
    }  
  }  
}  
  
# 2. which location  
provider "aws" {  
  region = var.region  
}  
  
# 3. what type of resource  
resource "aws_instance" "arsharaj-server" {  
  ami      = "ami-version"  
  instance_type = "t3.nano"  
  
  tags = {  
    Name = "arsharaj-server"  
  }  
}  
...
```

- Terraform config :
 - * It uses .tf extension
 - * Format is HCL (Hashicorp Config Language)
 - * Declarative Language
 - * Terraform supports json format also.
- Commands :
 - a. terraform init
 - b. terraform plan
 - c. terraform apply / destroy
- Benefits :
 1. consistency - define infrastructure in code
 2. automation - reduce manual work
 3. repeatable - replicate environments easily
- State Management :
 - * The state file (terraform.tfstate) maintains a detailed record of the current state of managed resources.
 - * This state file can be stored locally or remotely, with remote storage options enabling collaboration by sharing the state across teams and environments.
- Multiple resources using count, for each etc.

- Terraform Modules :
 - * Modules are containers for multiple resources that are used together.
 - * A module consists of a collection of .tf and .tf.json files kept together in a directory.
 - * Modules are the main way to package and reuse resource configurations with terraform.
- Terraform Workspace
- Terraform Cloud
 - * It is a managed service provided by hashicorp that facilitates collaboration on terraform configurations.
 - * Providing features like :
 1. remote state management
 2. version control system (vcs) integration
 3. automated runs
 4. secure variable management

DNS Setup

- DNS or domain name system is the internet service that translates human friendly domain names like `www.example.com` into machine readable ip addresses.

- Setting up apache web server.

1. `sudo yum install httpd`
2. `systemctl start/stop/status httpd`
3. `firewall-cmd --add-service=http --permanent`
4. `firewall-cmd --reload`
5. webserver config file under :
 - a. `/var/www/html/index.html`
 - b. `/etc/httpd/conf/httpd.conf`

- Setup our personal webpage.

- Enable firewall for http service.

- Setting up dns server.

1. `sudo yum install bind bind-utils` (berkeley internet name domain)
2. `systemctl start/stop/status named`
3. `firewall-cmd --add-service=dns --permanent`
4. `firewall-cmd --reload`
5. `dnf config` under :
 - a. `/etc/named.conf` - directory where all the zone files are present where you define hostname to ip.
 - b. `/var/named`

- Dns server config changes

- Verify dns changes

- DNS translate :

1. a record : hostname to ip
2. ptr record : ip to hostname
3. cname record : hostname to hostname

- Zones file :

- a. Forward zone - resolve domain to ip
- b. Reverse zone - resolve ip to domain

Understand SSL / TLS

- SSL stands for Secure Sockets Layer.

- TLS stands for Transport Layer Security.

- These are cryptographic protocols designed to provide secure communication over a computer network.

- TLS is the successor to SSL.

- SSL/TLS encrypt data and ensure its integrity, confidentiality and authenticity between a client and server.

- Key terms :

1. Confidentiality : data is only accessed by client/server.
2. Integrity : data is not modified in between.
3. Authentication : verifying the identity of the parties who they are supposed to be.

- Methods used :

1. Confidentiality -> Encryption
2. Integrity -> Hashing
3. Authentication -> Certificates

- Encryption : converting plain text information into a coded form (cipher text).

* Types of encryption :

1. Symmetric :

- Faster and more efficient, suitable for large data.
- Challenging key distribution, single key must be kept secret.

2. Asymmetric :

- Slower, suitable for small data.
- Simplified key distribution, secure even if the public key is shared.
 - a. encrypt - public key
 - b. decrypt - private key

3. Hybrid :

- combines the strength of both symmetric and asymmetric encryption to achieve efficient and secure communication.
- asymmetric encryption is used to securely exchange a symmetric key between parties.
- once the symmetric key is securely exchanged, it is used to encrypt and decrypt the actual data.
- symmetric encryption is faster and more efficient, making it ideal for encrypting large amounts of data.

* Algorithms :

1. Asymmetric : DSA, RSA, ECC, ECDH
2. Symmetric : AWS, 3DES, RC4

- Hashing : process of converting data into a fixed size string of characters, as a sequence of numbers and letters.

- Message Auth Code (MAC) for better hashing by combining message + code.

* Algorithms :

1. MD5 (Message Digest Algorithm 5) (128 bits)
2. SHA (Secure Hash Algorithm)
 - a. sha-1
 - b. sha-2/3 224 256 384 512
3. Hash based message authentication code (sha256hmac)

- Certificates :

* CA or Certificate Authority is a trusted organization that issues digital certificates to verify the identity of websites and enable secure, encrypted communication over the internet.

* CAs ensure the authenticity and integrity of the ssl certifications they provide.

* Formats for digital certificates :

1. PEM :

Encoding - base64
Extensions - .pem, .crt, .cer
Usage - web servers, email
Private key - no unless its a key file

2. DER :

Encoding - binary
Extensions - .der, .cer
Usage - java platforms, binary data handling
Private key - no

3. PKCS#7 :

Encoding - base64 or binary
Extensions - .p7b, .p7c
Usage - certificate chains
Private key - no

4. PKCS#12 :

Encoding - binary
Extensions - .p12, .pfx
Usage - export import certs with keys
Private key - yes

- Communication :
 - a. Client hello : hello, i support these ssl/tls versions and cipher suites.
 - b. Server hello : hello, i choose this ssl/tls version and cipher suites.
 - c. Server certificate : here is my certificate with my public key.
 - d. Certificate verification : i verify your certificate.
 - e. Key exchange :
 - Client - i encrypt a pre-master secret with your public key
 - Server - i decrypt the pre-master secret with my private key
 - f. Session key generation :
 - both - we generate a session key using the pre-master secret.
 - g. Client finished : handshake complete (encrypted with session key).
 - h. Server finished : handshake complete (encrypted with session key).
 - i. Secure communication : let's communicate securely using the session key.